

Advanced Optimization

Chapter 10 — Least-Squares Problems

Hongseok Kim

Department of Electronic Engineering, Sogang University

Fall 2026

Reading: Nocedal & Wright, *Numerical Optimization*, 2nd ed., Chapter 10

Least-squares problems

The objective has the special form

$$f(x) = \frac{1}{2} \sum_{j=1}^m r_j^2(x), \quad (10.1)$$

where each **residual** $r_j : \mathbb{R}^n \rightarrow \mathbb{R}$ is smooth, and $m \geq n$ throughout. With the residual vector $r(x) = (r_1(x), \dots, r_m(x))^T$ (10.2), $f(x) = \frac{1}{2} \|r(x)\|_2^2$.

The **Jacobian** $J(x)$ is the $m \times n$ matrix of first partial derivatives of the residuals:

$$J(x) = \left[\frac{\partial r_j}{\partial x_i} \right]_{\substack{j=1, \dots, m \\ i=1, \dots, n}} = \begin{bmatrix} \nabla r_1(x)^T \\ \vdots \\ \nabla r_m(x)^T \end{bmatrix}. \quad (10.3)$$

Gradient and Hessian

$$\nabla f(x) = \sum_{j=1}^m r_j(x) \nabla r_j(x) = J(x)^T r(x), \quad (10.4)$$

$$\nabla^2 f(x) = J(x)^T J(x) + \sum_{j=1}^m r_j(x) \nabla^2 r_j(x). \quad (10.5)$$

The distinctive feature of least squares

Given the Jacobian, the gradient (10.4) and the first term $J^T J$ of the Hessian come **without any second derivatives**.

$J^T J$ often dominates the second term: residuals small, or nearly affine, near the solution.

Example 10.1: fitting a model to data

Blood samples at times t_j , measured concentrations y_j . Model with parameters $x = (x_1, \dots, x_5)$:

$$\phi(x; t) = x_1 + tx_2 + t^2x_3 + x_4e^{-x_5t}. \quad (10.6)$$

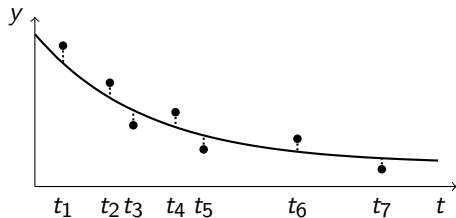


Figure 10.1 (redrawn)

Sum of the squared *vertical* distances,

$$\frac{1}{2} \sum_{j=1}^m [\phi(x; t_j) - y_j]^2, \quad (10.7)$$

which is (10.1) with

$$r_j(x) = \phi(x; t_j) - y_j. \quad (10.8)$$

Why the sum of squares?

$\|r(x)\|_\infty$ or $\|r(x)\|_1$ (10.11) lead to smooth *constrained* problems (Chapter 17); this chapter uses the ℓ_2 form (10.1).

Statistical motivation. Let $\epsilon_j = \phi(x; t_j) - y_j$ be i.i.d. with density g_σ . The likelihood of the observations is

$$p(y; x, \sigma) = \prod_{j=1}^m g_\sigma(\epsilon_j) = \prod_{j=1}^m g_\sigma(\phi(x; t_j) - y_j). \quad (10.12)$$

For a normal distribution, $p(y; x, \sigma) = (2\pi\sigma^2)^{-m/2} \exp(-\sum_j [\phi(x; t_j) - y_j]^2 / (2\sigma^2))$, so for fixed σ **the maximum likelihood estimate minimizes the sum of squares (10.7).**

Linear least-squares problems

If the model is linear in x , the residuals are linear: $r(x) = Jx - y$ with J , y independent of x ,

$$f(x) = \frac{1}{2} \|Jx - y\|^2, \quad \nabla f(x) = J^T (Jx - y), \quad \nabla^2 f(x) = J^T J. \quad (10.13)$$

f is convex, so any x^* with $\nabla f(x^*) = 0$ is a global minimizer (Theorem 2.5):

$$J^T J x^* = J^T y \quad (\text{the **normal equations**}). \quad (10.14)$$

Assume $m \geq n$ and J of full column rank.

Algorithm 1: normal equations. Form $J^T J$ and $J^T y$; Cholesky factorization $J^T J = \bar{R}^T \bar{R}$ (10.15); two triangular substitutions. Drawback: $\kappa(J^T J) = \kappa(J)^2$; the factorization can break down when J is ill conditioned.

Algorithm 2: QR

The Euclidean norm is unchanged by orthogonal transformations:

$\|Jx - y\| = \|Q^T(Jx - y)\|$ (10.16). QR factorization with column pivoting,

$$J\Pi = Q \begin{bmatrix} R \\ 0 \end{bmatrix} = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} R \\ 0 \end{bmatrix} = Q_1 R, \quad (10.17)$$

Π a permutation, Q orthogonal $m \times m$, R upper triangular $n \times n$. Then

$$\|Jx - y\|_2^2 = \|R(\Pi^T x) - Q_1^T y\|_2^2 + \|Q_2^T y\|_2^2, \quad (10.18)$$

and x affects only the first term: $x^* = \Pi R^{-1} Q_1^T y$ (solve $Rz = Q_1^T y$, permute).

The error is proportional to the condition number of J , not its square. Usually reliable.

Algorithm 3: SVD

$$J = U \begin{bmatrix} S \\ 0 \end{bmatrix} V^T = [U_1 \ U_2] \begin{bmatrix} S \\ 0 \end{bmatrix} V^T = U_1 S V^T, \quad (10.19)$$

U ($m \times m$) and V ($n \times n$) orthogonal, $S = \text{diag}(\sigma_1 \geq \cdots \geq \sigma_n > 0)$. As in (10.18), $\|Jx - y\|^2 = \|S(V^T x) - U_1^T y\|^2 + \|U_2^T y\|^2$ (10.20), hence

$$x^* = \sum_{i=1}^n \frac{u_i^T y}{\sigma_i} v_i. \quad (10.21)$$

- ▶ **Sensitivity.** Small σ_i : x^* is very sensitive to perturbations of $u_i^T y$.
- ▶ **Rank-deficient** J (some $\sigma_i = 0$): every x^* of the form (10.22), with arbitrary coefficients on those v_i , is a minimizer; zero coefficients give the smallest norm.

Choosing among the three; large problems

Method	Conditioning	When
Normal equations	$\kappa(J)^2$	$m \gg n$ or J sparse: store $J^T J$, not J
QR	$\kappa(J)$	usually reliable
SVD	$\kappa(J)$	most robust; sensitivity information; costliest

- ▶ Ill-conditioned J : omitting the terms with small σ_i from (10.21) gives an approximate solution that is less sensitive to perturbations.
- ▶ **Very large problems:** solve (10.14) by CG (Algorithm 5.2): one product with J and one with J^T per iteration, $J^T J$ never formed. LSQR (Paige and Saunders): same cost, better numerical properties.

Read: pp. 250–254.

The Gauss–Newton method

Newton's method with line search would solve $\nabla^2 f(x_k)p = -\nabla f(x_k)$.
Gauss–Newton solves instead

$$J_k^T J_k p_k^{\text{GN}} = -J_k^T r_k, \quad (10.23)$$

that is, it uses the approximation

$$\nabla^2 f_k \approx J_k^T J_k. \quad (10.24)$$

- ▶ **No second derivatives.** The individual residual Hessians $\nabla^2 r_j$ in (10.5) are not needed. If J_k is computed for the gradient anyway, (10.24) costs nothing more.
- ▶ **Often accurate.** When $J^T J$ dominates the second term of (10.5) near x^* (each $|r_j| \|\nabla^2 r_j\|$ small), the convergence rate is similar to Newton's.

Two more advantages

Descent. If J_k has full rank and $\nabla f_k \neq 0$, then by (10.4) and (10.23)

$$(p_k^{\text{GN}})^T \nabla f_k = (p_k^{\text{GN}})^T J_k^T r_k = -(p_k^{\text{GN}})^T J_k^T J_k p_k^{\text{GN}} = -\|J_k p_k^{\text{GN}}\|^2 \leq 0, \quad (10.25)$$

with equality only if $J_k p_k^{\text{GN}} = 0$, which means $J_k^T r_k = \nabla f_k = 0$. So p_k^{GN} is a descent direction, suitable for a line search (Armijo or Wolfe, (3.4), (3.6)).

A linear least-squares subproblem. (10.23) are the normal equations of

$$\min_p \frac{1}{2} \|J_k p + r_k\|^2. \quad (10.26)$$

So p_k^{GN} can be computed by QR or SVD of J_k , or by CG, **without forming $J_k^T J_k$** . (m large, n small: do not store J ; accumulate $J^T J$ and $J^T r$ residual by residual (10.27).)

Gauss–Newton as a linear model of the residuals

Subproblem (10.26) comes from the linear model $r(x_k + p) \approx r_k + J_k p$ substituted into $\frac{1}{2} \|\cdot\|^2$:

$$f(x_k + p) = \frac{1}{2} \|r(x_k + p)\|^2 \approx \frac{1}{2} \|J_k p + r_k\|^2,$$

and p_k^{GN} is the minimizer of this approximation.

Comparison with Chapter 6 (Not in the book.)

Both methods replace $\nabla^2 f_k$ by a matrix B_k built from first derivatives.

Quasi-Newton builds B_k from the history of gradient differences and must enforce $s_k^T y_k > 0$. Gauss–Newton takes $B_k = J_k^T J_k$ from the current Jacobian; it is positive semidefinite by construction.

Theorem 10.1: global convergence of Gauss–Newton

Uniform full-rank condition (10.28): $\|J(x)z\| \geq \gamma\|z\|$ on $\mathcal{N} \supset \mathcal{L} = \{x \mid f(x) \leq f(x_0)\}$.

Theorem 10.1

Suppose each residual function r_j is Lipschitz continuously differentiable in a neighborhood $\mathcal{N}$ of the bounded level set (10.29), and that the Jacobians $J(x)$ satisfy the uniform full-rank condition (10.28) on $\mathcal{N}$. Then if the iterates x_k are generated by the Gauss–Newton method with step lengths α_k that satisfy (3.6), we have

$$\lim_{k \rightarrow \infty} J_k^T r_k = 0.$$

Zoutendijk (Theorem 3.2) with $\cos \theta_k \geq \gamma^2/\bar{\beta}^2 > 0$ by (10.28). **Read:** proof, p. 256.

Local convergence of Gauss–Newton

Let $H(x)$ denote the second-order term $\sum_j r_j(x) \nabla^2 r_j(x)$ of (10.5), and write $J^T J(x)$ for $J(x)^T J(x)$. For a unit step, an argument like (3.31)–(3.33) gives

$$\|x_k + p_k^{\text{GN}} - x^*\| \approx \|[J^T J(x^*)]^{-1} H(x^*)\| \|x_k - x^*\| + O(\|x_k - x^*\|^2). \quad (10.30)$$

- ▶ If $\|[J^T J(x^*)]^{-1} H(x^*)\| \ll 1$, a unit Gauss–Newton step moves much closer to x^* : rapid local convergence.
- ▶ If $H(x^*) = 0$ (zero residual): quadratic convergence.
- ▶ Otherwise the rate is linear, with factor $\|[J^T J(x^*)]^{-1} H(x^*)\|$.

Read: p. 257.

Rank-deficient Jacobians; inexact Gauss–Newton

- ▶ If J_k is rank-deficient, (10.23) is singular but still solvable, because it is equivalent to (10.26); the solutions have the form (10.22).
- ▶ Then $\cos \theta_k$ is no longer bounded away from zero, and a result like Theorem 10.1 cannot be proved. (Levenberg–Marquardt removes this weakness.)
- ▶ **Inexact Gauss–Newton.** When n and m are large and J is sparse, factoring J_k or $J_k^T J_k$ at every iteration is expensive. Use the inexact Newton methods of Chapter 7 with $J_k^T J_k$ in place of $\nabla^2 f_k$; its positive semidefiniteness simplifies them.

The Levenberg–Marquardt method

Approximation (10.24) inside a **trust region**; rank-deficient J is no longer a difficulty:

$$\min_p \frac{1}{2} \|J_k p + r_k\|^2, \quad \text{subject to } \|p\| \leq \Delta_k, \quad (10.31)$$

that is, the model function of (4.3) is

$$m_k(p) = \frac{1}{2} \|r_k\|^2 + p^T J_k^T r_k + \frac{1}{2} p^T J_k^T J_k p. \quad (10.32)$$

Drop the index k . If $\|p^{\text{GN}}\| < \Delta$, then p^{GN} solves (10.31). Otherwise there is $\lambda > 0$ such that the solution $p = p^{\text{LM}}$ satisfies $\|p\| = \Delta$ and

$$(J^T J + \lambda I) p = -J^T r. \quad (10.33)$$

Lemma 10.2

Lemma 10.2

The vector p^{LM} is a solution of the trust-region subproblem

$$\min_p \|Jp + r\|^2, \quad \text{subject to } \|p\| \leq \Delta,$$

if and only if p^{LM} is feasible and there is a scalar $\lambda \geq 0$ such that

$$(J^T J + \lambda I) p^{\text{LM}} = -J^T r, \quad (10.34a)$$

$$\lambda(\Delta - \|p^{\text{LM}}\|) = 0. \quad (10.34b)$$

Theorem 4.1 applied; (4.8c) holds since $J^T J \succeq 0$, $\lambda \geq 0$. **Read:** proof, p. 259.

Computing the Levenberg–Marquardt step

(10.33) are the normal equations of a linear least-squares problem,

$$\min_p \frac{1}{2} \left\| \begin{bmatrix} J \\ \sqrt{\lambda} I \end{bmatrix} p + \begin{bmatrix} r \\ 0 \end{bmatrix} \right\|^2, \quad (10.35)$$

so the step is computed without forming $J^T J$. Find λ from Δ by rootfinding (Ch. 4).

QR for several λ . Householder QR of J once (10.37); then, for each λ , $n(n+1)/2$

Givens rotations:

$$\begin{bmatrix} R_\lambda \\ 0 \end{bmatrix} = Q_\lambda^T \begin{bmatrix} J \\ \sqrt{\lambda} I \end{bmatrix}, \quad R_\lambda^T R_\lambda = J^T J + \lambda I. \quad (10.36)$$

Cost: $O(mn^2)$ once, then $O(n^3)$ per value of λ . **Read:** pp. 259–260.

Scaling; large sparse problems

- ▶ Least-squares problems are often poorly scaled: some variables of size 10^4 , others 10^{-6} . Use an **ellipsoidal** trust region with a diagonal D_k :

$$\min_p \frac{1}{2} \|J_k p + r_k\|^2, \quad \text{subject to } \|D_k p\| \leq \Delta_k, \quad (10.39)$$

$$(J_k^T J_k + \lambda D_k^2) p_k^{\text{LM}} = -J_k^T r_k. \quad (10.40)$$

Seber and Wild: match the diagonal of D_k^2 to that of $J_k^T J_k$, which makes the algorithm invariant under diagonal scaling of x (cf. Section 4.5).

- ▶ **Large sparse J :** solve (10.31) or (10.39) approximately by CG–Steihaug (Algorithm 7.2) with $J_k^T J_k$. No negative curvature can arise, and only products with J_k and J_k^T are needed. **Read:** pp. 260–261.

Theorem 10.3: global convergence of Levenberg–Marquardt

Theorem 10.3

Let $\eta \in (0, \frac{1}{4})$ in Algorithm 4.1 of Chapter 4, and suppose that the level set $\mathcal{L}$ defined in (10.29) is bounded and that the residual functions $r_j(\cdot)$, $j = 1, 2, \dots, m$ are Lipschitz continuously differentiable in a neighborhood $\mathcal{N}$ of $\mathcal{L}$. Assume that for each k , the approximate solution p_k of (10.31) satisfies the inequality

$$m_k(0) - m_k(p_k) \geq c_1 \|J_k^T r_k\| \min \left(\Delta_k, \frac{\|J_k^T r_k\|}{\|J_k^T J_k\|} \right), \quad (10.42)$$

for some constant $c_1 > 0$, and in addition $\|p_k\| \leq \gamma \Delta_k$ for some constant $\gamma \geq 1$. We then have that $\lim_{k \rightarrow \infty} \nabla f_k = \lim_{k \rightarrow \infty} J_k^T r_k = 0$.

Theorem 4.6 applied; (10.42) is the Cauchy-point decrease. **Read:** proof, p. 262.

Local behavior; large-residual problems

Near a solution x^* where $J^T J$ dominates, the trust region becomes inactive and the algorithm takes Gauss–Newton steps: the local rate is again (10.30).

Large-residual problems

If the second-order part of $\nabla^2 f$ is too large to ignore, the model (10.31) is inadequate. Gauss–Newton and Levenberg–Marquardt then converge only **linearly**, slower than Newton or quasi-Newton methods.

- ▶ If the $\nabla^2 r_j$ are easy to compute: Newton's method with a trust region or line search. Otherwise quasi-Newton (superlinear). But both may be worse than Gauss–Newton in the early iterations.
- ▶ Large residuals at the solution often mean the model is inadequate or the data are wrong.

Hybrid methods

We rarely know in advance whether the residuals at the solution will be small or large. Hybrid algorithms behave like Gauss–Newton when they are small, and switch to Newton or quasi-Newton steps otherwise.

- ▶ **Fletcher and Xu.** If the Gauss–Newton step reduces f by a fixed factor (say 5), take it and set $B_k \leftarrow J_k^T J_k$; otherwise take a line-search step along $-B_k^{-1} \nabla f_k$. Then apply a BFGS-like update to B_k . Zero residual: eventually Gauss–Newton (quadratic). Nonzero residual: eventually BFGS (superlinear).
- ▶ **Dennis, Gay, and Welsch (NL2SOL).** Keep $J_k^T J_k$; approximate the rest:

$$B_k = J_k^T J_k + S_k, \quad S_k \approx \sum_{j=1}^m r_j(x_k) \nabla^2 r_j(x_k).$$

The Dennis–Gay–Welsch update

Each $(B_j)_{k+1} \approx \nabla^2 r_j$ should mimic the change of ∇r_j over the step; weighted by $r_j(x_{k+1})$ and summed, a **secant equation** for S_{k+1} :

$$S_{k+1}(x_{k+1} - x_k) = J_{k+1}^T r_{k+1} - J_k^T r_{k+1}.$$

With symmetry and closeness to S_k (as in Chapter 6),

$$S_{k+1} = S_k + \frac{(y^\sharp - S_k s)y^T + y(y^\sharp - S_k s)^T}{y^T s} - \frac{(y^\sharp - S_k s)^T s}{(y^T s)^2} yy^T, \quad (10.43)$$

$$s = x_{k+1} - x_k, \quad y = J_{k+1}^T r_{k+1} - J_k^T r_{k+1}, \quad y^\sharp = J_{k+1}^T r_{k+1} - J_k^T r_{k+1}.$$

A variant of DFP (identical if $y^\sharp = y$). S_k is scaled down before each update, so it can vanish on zero-residual problems. **Read:** pp. 263–265.

Orthogonal distance regression

Errors in the t_j as well (*errors-in-variables*): perturbations δ_j ,

$$y_j = \phi(x; t_j + \delta_j) + \epsilon_j, \quad j = 1, 2, \dots, m, \quad (10.44)$$

$$\min_{x, \delta_j, \epsilon_j} \frac{1}{2} \sum_{j=1}^m w_j^2 \epsilon_j^2 + d_j^2 \delta_j^2, \quad \text{subject to (10.44)}. \quad (10.45)$$

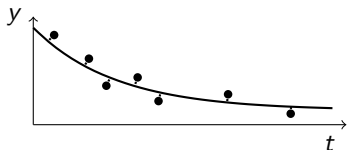


Figure 10.2 (redrawn). With $w_j = d_j$, each term of (10.45) is the shortest distance from (t_j, y_j) to the curve; the shortest path meets the curve orthogonally. Linear case: *total least squares*.

ODR as a least-squares problem with structure

Eliminate ϵ_j by (10.44): unconstrained, $2m$ residuals, $n + m$ unknowns,

$$\min_{x, \delta} F(x, \delta) = \frac{1}{2} \sum_{j=1}^m w_j^2 [y_j - \phi(x; t_j + \delta_j)]^2 + d_j^2 \delta_j^2 = \frac{1}{2} \sum_{j=1}^{2m} r_j^2(x, \delta). \quad (10.46)$$

Since $\partial r_j / \partial \delta_i = 0$ for $i \neq j$ and $\partial r_{m+j} / \partial x_i = 0$,

$$J(x, \delta) = \begin{bmatrix} \hat{J} & V \\ 0 & D \end{bmatrix}, \quad (10.48)$$

V, D diagonal; $\hat{J}$ the $m \times n$ Jacobian of $w_j \phi(t_j + \delta_j; x)$ in x . In (10.33) partitioned as (10.49), the block $V^2 + D^2 + \lambda I$ is diagonal: eliminate p_δ , solve an $n \times n$ system for p_x . **Read:** pp. 265–267.

Perspectives and software

- ▶ The original Levenberg–Marquardt algorithm adjusted λ in (10.33) directly, increasing or decreasing it by a factor according to whether the previous step decreased f . The connection with trust regions was established by Moré.
- ▶ **Inexact Levenberg–Marquardt** (Wright and Holt): steps $\bar{p}_k$ with
$$\|(J_k^T J_k + \lambda_k I)\bar{p}_k + J_k^T r_k\| \leq \eta_k \|J_k^T r_k\|, \quad \eta_k \in [0, \eta],$$

a forcing sequence as in (7.3). Implemented with LSQR, which gives approximate solutions of (10.35) for several λ_k at once.

- ▶ Software: IMSL, HSL, NAG, SAS; DFNLP, MINPACK, NL2SOL, NLSSOL; LANCELOT, KNITRO, SNOPT (large scale); ORDPACK (ODR). All accept Jacobians or compute them by finite differencing.

Two uses of this chapter (Not in the book.)

Power system state estimation

Weighted least squares $\min_x \frac{1}{2}(z - h(x))^T W(z - h(x))$: measurements z , power-flow equations h . The standard algorithm is Gauss–Newton with weights: the “gain matrix” $H^T W H$ is $J^T J$ of (10.23), observability is the rank condition (10.28), and bad-data detection is residual analysis.

Machine learning

With a squared-error loss, training is (10.1) and $J^T J$ is the Gauss–Newton matrix; for other losses the same construction is called generalized Gauss–Newton. The Shampoo preconditioner can be viewed as a Kronecker approximation of the Gauss–Newton part of the Hessian (Morwani et al., ICLR 2025): see the November presentations.

Chapter 10 in one table

Method	Hessian model	Globalization	Local rate
Gauss–Newton	$J^T J$	line search	(10.30); quadratic if $H(x^*) = 0$
Levenberg–Marquardt	$J^T J$	trust region	same as Gauss–Newton
Fletcher–Xu hybrid	$J^T J$ or BFGS	line search	quadratic or superlinear
Dennis–Gay–Welsch	$J^T J + S_k$	trust region	superlinear
Newton	(10.5)	either	quadratic

- ▶ Small residuals: Gauss–Newton or Levenberg–Marquardt; the rate is nearly quadratic.
- ▶ Large residuals: their rate is linear; use Newton, quasi-Newton, or a hybrid.
- ▶ Rank-deficient or ill-conditioned J : Levenberg–Marquardt, not Gauss–Newton.

Three results to carry forward

1. **$J^T J$ is the part of the Hessian that comes with the gradient.** Its quality as an approximation is governed by the residuals, (10.5) and (10.30): small residuals, fast convergence; large residuals, linear convergence.
2. **Gauss–Newton is Newton's method with $B_k = J_k^T J_k$; Levenberg–Marquardt is the same model in a trust region.**
 $(J^T J + \lambda I)p = -J^T r$ is the trust-region step (Lemma 10.2), and the global theory is Chapter 3 and Chapter 4 applied.
3. **Never form $J^T J$ unless $m \gg n$.** The step solves a linear least-squares problem, (10.26) or (10.35), by QR, SVD or CG/LSQR on J itself.

Proved in the book: Theorem 10.1, Lemma 10.2, Theorem 10.3. Derivation only: (10.30).

Homework (problems 4–5 are not in the book)

1. Exercise 10.1(a): J ($m \times n$, $m \geq n$) has full column rank if and only if $J^T J$ is nonsingular.
2. Exercise 10.2: show that the function $f(x)$ in (10.13) is convex.
3. Exercise 10.3(a): if Q is orthogonal, then $\|Qx\| = \|x\|$ for any vector x .
4. By hand. $n = 1$, $m = 2$, $r_1(x) = x - 2$, $r_2(x) = x^2 - 1$, $x_0 = 0$. Compute the Gauss–Newton step (10.23), the Levenberg–Marquardt step (10.33) with $\lambda = 1$, and the Newton step. Which of the three are descent directions, and why?
5. Implement Gauss–Newton with your Chapter 3 line search for the model (10.6): generate data from $x = (1, 0, 0, 2, 1)$ at $t_j = 1, \dots, 7$ with small Gaussian noise, and fit from $x_0 = (0, 0, 0, 1, 0.5)$. Compare the iterates with `scipy.optimize.least_squares`.